

Markov Process and Optimality Proofs

WHAT IS REINFORCEMENT LEARNING?

The Core Idea:

- Agent learns by interacting with environment
- Receives feedback (rewards) for actions
- Adapts behavior to maximize cumulative reward
- No prior knowledge of system rules

Key Components:

1. **Agent:** Decision maker
2. **Environment:** Everything agent doesn't control
3. **State:** Current information available
4. **Action:** What agent can do
5. **Reward:** Numerical feedback

The Autonomous Car Analogy:

- Driver (agent) learns from experience
- Car's position, speed, sensors = state
- Pressing gas/brake/steering = actions
- Reaching destination safely = reward



THE MARKOV PROPERTY - To predict the future, all you need is the present.

The Memoryless Assumption:

- Future depends ONLY on current state
- Past doesn't matter (history irrelevant)
- Formal: $P(s_{t+1} | s_t, a_t) = P(s_{t+1} | s_t, a_t, s_{t-1}, \dots, s_0)$

Real-World Analogy:

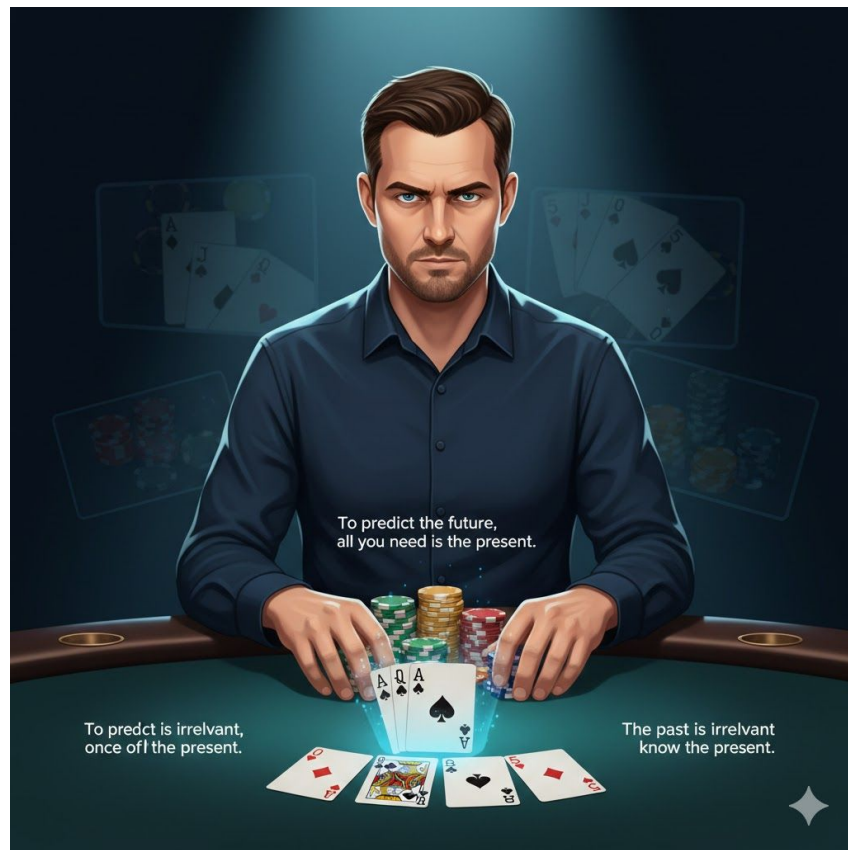
- **Card Game:** Your next hand depends only on what's on the table now, not how you got here
- **Weather:** Tomorrow's weather depends on today's weather, not the entire weather history
- **Chess:** Best next move depends on current board position, not move history
- **Investment:** Future returns depend on current portfolio, not past trades

Why It Matters:

- Makes problems computationally tractable
- Allows using past data to learn transition probabilities
- Enables dynamic programming solutions
- Foundation for all RL algorithms

Mathematical Statement:

- $P(S_{t+1} = s' | S_t = s, A_t = a) = P(S_{t+1} = s' | \text{entire history})$



MARKOV'S ASSUMPTION

The Formal Statement:

- "Given the present state, the future is conditionally independent of the past"
- What This Means:
- Current state summarizes all relevant history
- State is a sufficient statistic for the future
- Past trajectory doesn't add information beyond current state

Mathematical Formulation:

- $P(S_{t+1}, R_{t+1} \mid S_t, A_t, S_{t-1}, A_{t-1}, \dots, S_0) = P(S_{t+1}, R_{t+1} \mid S_t, A_t)$

In Practice:

- We design states to be "Markovian"
- Include all information needed for decision-making
- Balance completeness vs computational tractability

Example - Robot State:

-  Non-Markovian: Position only (can't infer velocity)
-  Markovian: Position + velocity (complete dynamics)

MARKOV CHAINS

What Is a Markov Chain?

- A sequence of random states
- Transitions follow Markov Property
- **No actions** (no control/decisions)
- Fully defined by transition probabilities

State Transition Matrix:

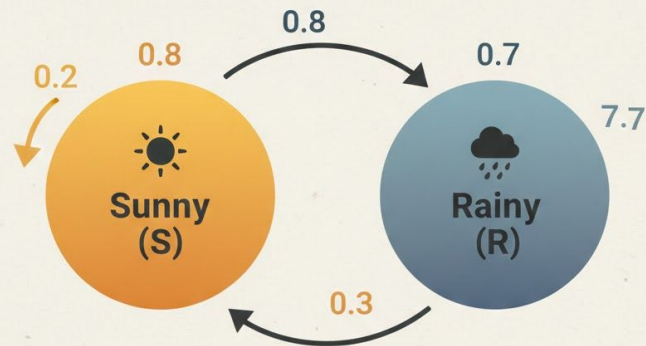
- $P = [P_{ij}]$ where $P_{ij} = P(S_{t+1} = j \mid S_t = i)$
- Each row sums to 1 (must go somewhere)
- Represents "if we're in state i , where might we go?"

Real-World Examples:

- Weather: Sunny $\rightarrow$ Cloudy $\rightarrow$ Rainy $\rightarrow$ Sunny
- Customer behavior: Browsing $\rightarrow$ Cart $\rightarrow$ Purchase $\rightarrow$ Churn
- Stock market: Bull $\rightarrow$ Neutral $\rightarrow$ Bear
- Disease progression: Healthy $\rightarrow$ Infected $\rightarrow$ Recovered

Key Properties:

- Stationary distribution: Long-run probability of each state
- Multi-step transitions: P^n gives n -step probabilities



Transition matrix (Today $\rightarrow$ Tomorrow)

Today S	Tomorrow: S R	
	S	R
0.8 0.2	0.3 2	0.3.7

Markov property: tomorrow depends only on today. No actions.

MARKOV DECISION PROCESS (MDP)

Definition:

- A Markov Chain WITH actions
- Agent chooses action → environment responds
- Formally: (S, A, P, R, γ)

Five Components:

1. **S:** State space (all possible situations)
2. **A:** Action space (what agent can do)
3. **P:** Transition probabilities $P(s' | s, a)$
4. **R:** Reward function $R(s, a)$ or $R(s, a, s')$
5. **γ :** Discount factor ($0 < \gamma \leq 1$)

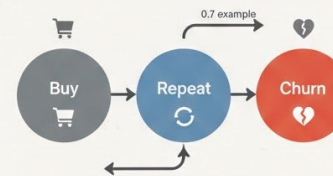
MDP Flow:

- State $s_t \rightarrow$ Action $a_t \rightarrow$ Reward r_t + Next State $s_{t+1} \rightarrow$ Repeat

Real-World Example:

- State: Robot position + battery level
- Action: Move left/right/stay/charge
- Transition: Where robot ends up (stochastic)
- Reward: +10 for goal, -1 per step, -5 for obstacle
- Discount: $\gamma = 0.99$ (prefer rewards sooner)

Markov Chain (no actions)



Customer behavior evolves on it own.

MDP (actions influence transitions)



You choose actions that affect what happens next.

Formal Definition - MDP

A Markov Decision Process is a tuple (S, A, P, R, γ) :

1. State Space (S):

- The set of all possible states the agent can be in. Can be:
- Discrete: $\{\text{State1}, \text{State2}, \dots, \text{StateN}\}$
- Continuous: Real-valued vectors (e.g., robot position in $\mathbb{R}^3$)
- Examples:
 - Chess: $S =$ all legal board positions ($\sim 10^{43}$ states)
 - Robotics: $S = (x, y, \theta, \text{battery_level}) \in \mathbb{R}^4$
 - Trading: $S = (\text{prices}, \text{holdings}, \text{time}) \in \mathbb{R}^n$

2. Action Space (A):

- The set of actions the agent can take. Can be:
- Discrete: Finite set like $\{\text{Up}, \text{Down}, \text{Left}, \text{Right}\}$
- Continuous: Real-valued vectors (e.g., steering angle $\in [-30^\circ, +30^\circ]$)
- Examples:
 - Chess: $A =$ all legal moves from current position
 - Robotics: $A =$ motor torques $\in \mathbb{R}^k$
 - Trading: $A = \{\text{Buy}, \text{Sell}, \text{Hold}\} \times [\text{quantity}]$

3. Transition Probability (P):

- $P(s' | s, a)$ = probability of reaching state s' after taking action a in state s .
- This captures the environment's dynamics.
- Example - Robot Navigation:
 - $P((0,1) | (0,0), \text{Up}) = 0.8$ [successfully moves up]
 - $P((0,0) | (0,0), \text{Up}) = 0.1$ [slips, stays in place]
 - $P((1,0) | (0,0), \text{Up}) = 0.1$ [slips sideways]

Formal Definition - MDP

4. Reward Function (R):

- $R(s, a)$ or $R(s, a, s')$ gives the immediate reward.
- Two common formulations:
 - $R(s, a)$: Reward depends on state and action
 - $R(s, a, s')$: Reward depends on transition
- This encodes the goal - what we want to maximize.
- Examples:
 - Game: +1 for win, -1 for loss, 0 otherwise
 - Robot: +100 for reaching goal, -1 per step, -50 for collision
 - Trading: Profit/loss from transaction

5. Discount Factor (γ):

- A number between 0 and 1 that weights future rewards:
- Reward today = r
 - Same reward tomorrow = γr
 - Same reward in n steps = $\gamma^n r$
- Intuition:
 - γ close to 0: Myopic (only care about immediate rewards)
 - γ close to 1: Far-sighted (value future rewards highly)
 - $\gamma = 1$: All rewards equally important (requires episodic tasks)
- Why Discount?
 - Multiple justifications:
 - Financial: Money now is worth more than money later (time value)
 - Uncertainty: Future is uncertain; world might end
 - Mathematical: Ensures infinite sums converge
 - Computational: Makes problems tractable
- Concrete Example - Grid World:
 - A robot navigating a 4x4 grid:
 - $S: \{(0,0), (0,1), \dots, (3,3)\} = 16$ states
 - $A: \{\text{Up, Down, Left, Right}\}$
 - P : Intended direction with 0.8 probability, perpendicular directions with 0.1 each
 - R : +10 for reaching (3,3), -10 for (3,1) pit, -1 for each step
 - γ : 0.9

MDP Dynamics

At each timestep t :

1. Agent observes state s_t
2. Agent chooses action a_t based on policy π
3. Environment samples: $s_{t+1} \sim P(\cdot | s_t, a_t)$
4. Environment returns reward: $r_t = R(s_t, a_t)$
5. Episode continues from s_{t+1}

The sequence $(s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ is called a **trajectory or episode**.

MDP vs BANDIT PROBLEM

Multi-Armed Bandit Problem:

- Single state (stateless)
- Choose among k actions (arms)
- Each action gives random reward
- Goal: Maximize cumulative reward
- No state transitions

Markov Decision Process:

- Multiple states
- Actions affect future states
- Sequential decision-making
- State transitions matter
- Long-term consequences

Aspect	Bandit	MDP
States	1 (stateless)	Multiple (S)
Actions affect future?	No	Yes
Complexity	Exploration-exploitation only	+ Sequential planning
Horizon	Single-step rewards	Multi-step cumulative
Example	Slot machines	Chess, robotics

POLICY

What Is a Policy?

- Mapping from states to actions
- The agent's "strategy" or "behavior"
- Answers: "What should I do in this situation?"

Two Types:

Deterministic Policy:

- $\pi(s) = a$
- Given state s , always choose action a
- Example: "At red light, always stop"

Stochastic Policy:

- $\pi(a | s)$ = probability of action a in state s
- Given state s , choose action a with some probability
- Example: "At yellow light: stop 70%, go 30%"

Policy (π): A rule for what to do in each situation

Like a sports playbook or a cooking recipe

Deterministic Policy



Same state $\rightarrow$ same action

Stochastic Policy



$\pi(\text{Hungry_at downtown})$:



Same state $\rightarrow$ different actions
with probabilities

GOAL OF MDP

Primary Goal:

- Find policy π^* that maximizes expected cumulative discounted reward
- Return (Total Reward):
- $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$

Objective Function:

- $\pi^* = \operatorname{argmax}_{\pi} E[G_t | \pi]$

What This Means:

- Not just maximize immediate reward
- Maximize long-term cumulative reward
- Balance short-term vs long-term gains
- Account for uncertainty (expectation)

Two Formulations:

1. **Episodic:** Maximize return per episode
2. **Continuing:** Maximize average or discounted infinite-horizon return

Example Goals:

- Chess: Maximize probability of winning
- Robot: Reach goal in minimum steps
- Trading: Maximize cumulative profit

Immediate reward vs long-term return

Student chooses: Party tonight or Study tonight



Action: Party

- Immediate reward: **+10 happiness**
- Future consequence: poor exam **(-20)**
- Total return: **10 + (-20) = -10**



Action: Study

- Immediate reward: **+2 happiness**
- Future consequence: good exam **(+30)**
- Total return: **2 + 30 = +32**

Optimal policy: Study (sacrifice now for future gain)

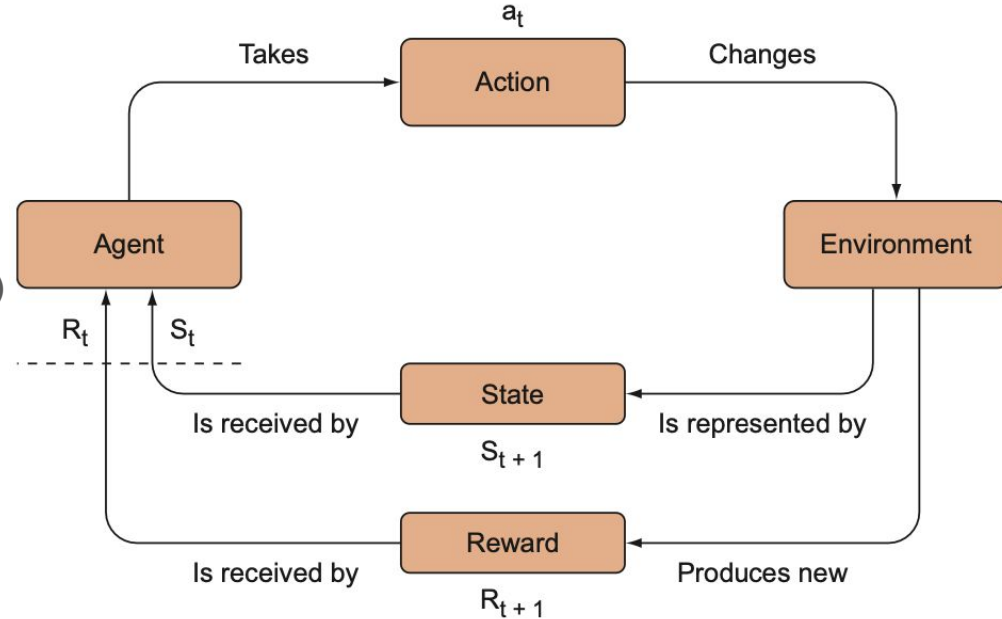
AGENT-ENVIRONMENT INTERFACE

The Cycle:

1. Agent observes state s_t
2. Agent selects action a_t based on policy π
3. Environment transitions to $s_{t+1} \sim P(\cdot | s_t, a_t)$
4. Environment returns reward $r_t = R(s_t, a_t)$
5. Repeat from step 1

Boundary:

- **Agent:** Everything inside the decision-maker
- **Environment:** Everything outside agent's control



FINITE vs INFINITE MDP

Finite MDP:

- Finite state space: $|S| < \infty$
- Finite action space: $|A| < \infty$
- Can store values in tables
- Examples: Chess, grid world, tic-tac-toe

Infinite MDP:

- Infinite or very large state/action space
- Continuous spaces: $S \subseteq \mathbb{R}^n, A \subseteq \mathbb{R}^m$
- Requires function approximation
- Examples: Robot control, autonomous driving

Aspect	Finite	Infinite
State space	Discrete, small	Continuous, large
Storage	Tabular	Function approximation
Algorithms	Value/Policy iteration	Neural networks, linear functions
Convergence	Guaranteed (under conditions)	Approximate
Examples	Games, puzzles	Control, robotics

ASSUMPTIONS FOR MDP

1. Markov Property (Most Critical)

- Future depends only on current state
- $P(s_{t+1} \mid s_t, a_t, \text{history}) = P(s_{t+1} \mid s_t, a_t)$

2. Stationary Dynamics

- Transition probabilities don't change over time
- $P(s'|s,a)$ is constant

3. Stationary Rewards

- Reward function doesn't change
- $R(s,a)$ is constant over time

4. Known State

- Agent observes true state (full observability)
- Or: Well-defined observation function

5. Well-Defined Action Space

- Available actions are known in each state
- Actions are executable

VALUE-BASED vs POLICY-BASED RL

Value-Based Approach:

- Learn value functions $V(s)$ or $Q(s,a)$
- Derive policy from value: $\pi(s) = \operatorname{argmax}_a Q(s,a)$
- Examples: Q-learning, SARSA, DQN
- Process: Experience $\rightarrow$ Values $\rightarrow$ Policy

Policy-Based Approach:

- Learn policy directly $\pi_{\theta}(a|s)$
- Optimize policy parameters θ
- Examples: REINFORCE, PPO, TRPO
- Process: Experience $\rightarrow$ Policy

Aspect	Value-Based	Policy-Based
What learned	$V(s)$ or $Q(s,a)$	$\pi(a s)$
Exploration	ϵ -greedy needed	Built into sampling
Continuous actions	Difficult	Natural
Convergence	Can be unstable	Generally stable
Sample efficiency	High	Low

VALUE FUNCTIONS

What Is Value?

- Expected cumulative reward from a state
- How "good" is a state (or state-action pair)?
- Depends on policy being followed

State Value Function $V^\pi(s)$:

- $V^\pi(s) = E_\pi[G_t | S_t = s]$ = Expected return starting from s , following policy π

State-Action Value Function $Q^\pi(s,a)$:

$Q^\pi(s,a) = E_\pi[G_t | S_t = s, A_t = a]$ = Expected return from s , taking action a , then following π

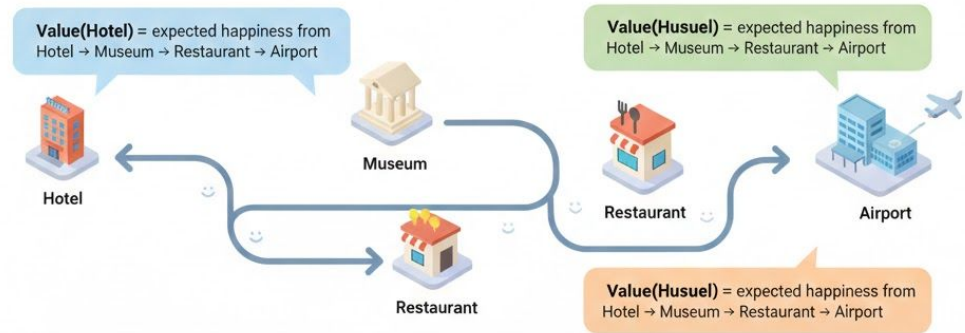
Relationship:

- $V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s,a)$

Optimal Value Functions:

- $V^*(s) = \max_\pi V^\pi(s)$
- $Q^*(s,a) = \max_\pi Q^\pi(s,a)$
- Optimal policy: $\pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$

Value = Future Happiness



BELLMAN EQUATION

The Recursive Insight:

- Value of state = **immediate reward + discounted value of next state**
- Bellman Equation for V^π :
- $V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V^\pi(s')]$

Bellman Equation for Q^π :

- $Q^\pi(s,a) = \sum_{s'} P(s'|s,a) [R(s,a) + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a')]$

Intuition:

- Current value = immediate payoff + future value
- Turns infinite sum into recursive relationship
- Foundation of ALL RL algorithms

Why Important:

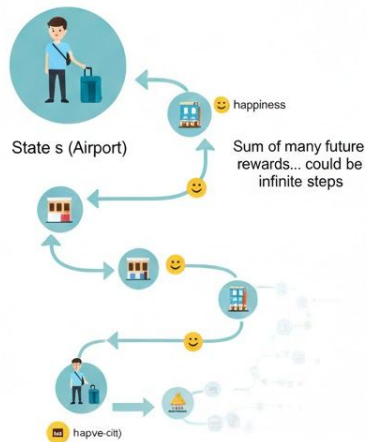
- Enables iterative value computation
- Bootstrapping: Update estimates using other estimates
- Allows learning from single transitions

Bellman Equation: City Travel Analogy

$$\text{Value}(\text{airport}) = \text{Reward}(\text{airport}) + \gamma \times \text{Value}(\text{next_location})$$

This recursion is the Bellman equation.
(γ = discount factor, $0 \leq \gamma < 1$)

Naive Approach



Smart Approach (Bellman)



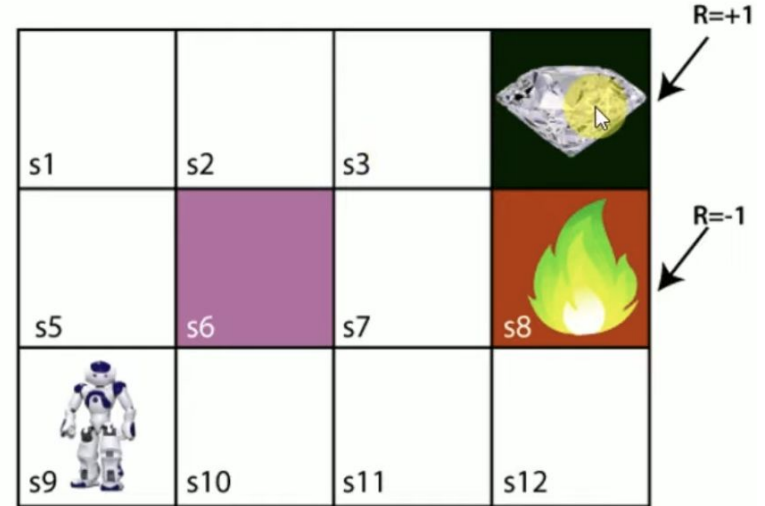
BELLMAN EQUATION - Numerical

- The Bellman equation can be written as:

$$V(s) = \max[R(s, a) + \gamma V(s')]$$

- Where,
- $V(s)$ = value calculated at a particular point.
- $R(s, a)$ = Reward at a particular state s by performing an action a .
- γ = Discount factor
- $V(s')$ = The value at the previous state.
- In the above equation, we are taking the max of the complete values because the agent tries to find the optimal solution always.

System Settings



BELLMAN OPTIMALITY EQUATION

From Policy Evaluation to Optimization:

- Previous: $V^\pi(s)$ for given policy π
- Now: $V^*(s)$ for the BEST policy
- Bellman Optimality for V^*
- $V^*(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V^*(s')]$

Bellman Optimality for Q^* :

- $Q^*(s,a) = \sum_{s'} P(s'|s,a) [R(s,a) + \gamma \max_{a'} Q^*(s',a')]$

Key Change:

- Bellman equation: Weighted sum over actions ($\sum_a \pi(a|s)$)
- Bellman optimality: Max over actions ($\max_a$)

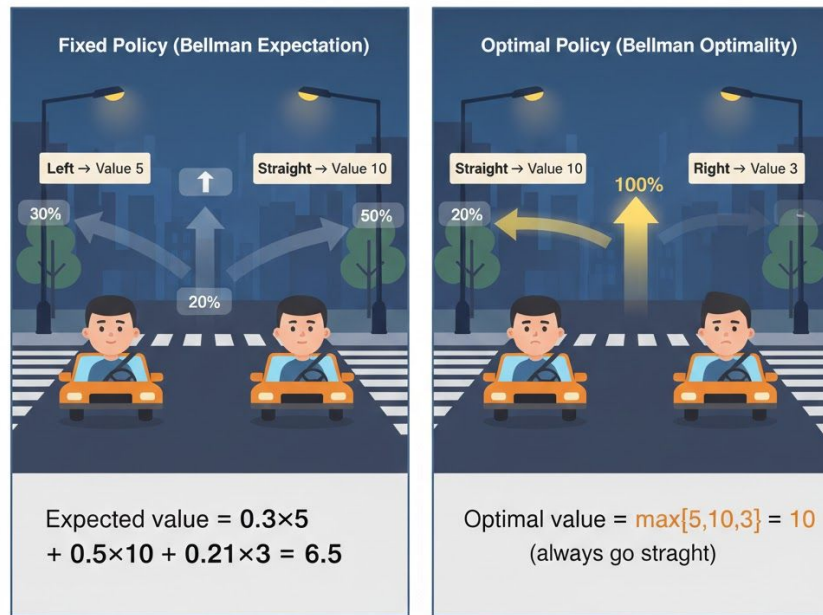
Optimal Policy:

- $\pi^*(s) = \operatorname{argmax}_a Q^*(s,a) = \operatorname{argmax}_a \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V^*(s')]$

Properties:

- Unique solution V^* , Q^* (optimal values are unique)
- Optimal policy may not be unique (ties possible)
- Foundation of value iteration and Q-learning

Bellman Equation: Expectation vs. Optimality



Expectation evaluates a given policy; optimality takes the max over actions.